

UNIT III

System Model: Deadlock Characterization – Methods for handling Deadlocks -Deadlock Prevention – Deadlock avoidance – Deadlock detection – Recovery from Deadlocks - Storage Management – Swapping – Contiguous Memory allocation – Paging – Segmentation – Segmentation with Paging -

2 Marks

1. Define deadlock. (NOV 13, NOV '15)

A process requests resources; if the resources are not available at that time, the process enters a wait state. Waiting processes may never again change state, because the resources they have requested are held by other waiting processes. This situation is called a deadlock.

2. What is the sequence in which resources may be utilized?

Under normal mode of operation, a process may utilize a resource in the following sequence:

- Request: If the request cannot be granted immediately, then the requesting process must wait until it can acquire the resource.
- Use: The process can operate on the resource.
- Release: The process releases the resource.

3. What are conditions under which a deadlock situation may arise? Or Write four general strategies for dealing with deadlocks? (APR'15)(NOV'14)

A deadlock situation can arise if the following four conditions hold simultaneously in a system:

- Mutual exclusion
- Hold and wait
- No pre-emption
- Circular-wait

4. What is a resource-allocation graph? (APR'12) (APR'15) (APR'17)

Deadlocks can be described more precisely in terms of a directed graph called a system resource allocation graph. This graph consists of a set of vertices V and a set of edges E . The set of vertices V is partitioned into two different types of nodes; P the set consisting of all active processes in the system and R the set consisting of all resource types in the system.

5. Define request edge and assignment edge.

A directed edge from process P_i to resource type R_j is denoted by P_i, R_j ; it signifies that process P_i requested an instance of resource type R_j and is currently waiting for that resource. A directed edge from resource type R_j to process P_i is denoted by R_j, P_i , it signifies that an instance of resource type has been allocated to a process P_i . A directed edge P_i, R_j is called a request edge. A directed edge R_j, P_i is called an assignment edge.

6. What are the methods for handling deadlocks? (or)

Write down three ways to deal with deadlock problem. (Nov' 2017)

The deadlock problem can be dealt with in one of the three ways:

- Use a protocol to prevent or avoid deadlocks, ensuring that the system will never enter a deadlock state.
- Allow the system to enter the deadlock state, detect it and then recover.
- Ignore the problem all together, and pretend that deadlocks never occur in the system.

7. Define deadlock prevention.

Deadlock prevention is a set of methods for ensuring that at least one of the four necessary conditions like mutual exclusion, hold and wait, no preemption and circular wait cannot hold. By ensuring that that at least one of these conditions cannot hold, the occurrence of a deadlock can be prevented.

8. Define deadlock avoidance.

An alternative method for avoiding deadlocks is to require additional information about how resources are to be requested. Each request requires the system consider the resources currently available, the resources currently allocated to each process, and the future requests and releases of each process, to decide whether the could be satisfied or must wait to avoid a possible future deadlock.

9. What are a safe state and an unsafe state?

A state is safe if the system can allocate resources to each process in some order and still void a deadlock. A system is in safe state only if there exists a safe sequence. A sequence of processes $\langle P_1, P_2, \dots, P_n \rangle$ is a safe sequence for the current allocation state if, for each P_i , the resource that P_i can still request can be satisfied by the current available resource plus the resource held by all the P_j , with $j < i$. if no such sequence exists, then the system state is said to be unsafe.

10. What is banker's algorithm?

Banker's algorithm is a deadlock avoidance algorithm that is applicable to a resource allocation system with multiple instances of each resource type. The two algorithms used for its implementation are:

- Safety algorithm: The algorithm for finding out whether or not a system is in a safe state.
- Resource-request algorithm: if the resulting resource allocations safe, the transaction is completed and process P_i is allocated its resources. If the new state is unsafe P_i must wait and the old resource-allocation state is restored.

11. Define logical address and physical address.

An address generated by the CPU is referred as logical address. An address seen by the memory unit that is the one loaded into the memory address register of the memory is commonly referred to as physical address.

12. What is logical address space and physical address space?

The set of all logical addresses generated by a program is called a logical address space; the set of all physical addresses corresponding to these logical addresses is a physical address space.

13. Why the page sizes is always power of 2 in memory? (NOV 13)

In operating systems that paging is implemented by breaking up an address into a page and offset number. It is most efficient to break the address into X page bits and Y offset bits, rather than perform arithmetic on the address to calculate the page number and offset. Because each bit position represents a power of 2, splitting an address between bits results in a page size that is a power of 2.

14. What is the main function of the memory-management unit?

The runtime mapping from virtual to physical addresses is done by a hardware device called a memory management unit (MMU).

15. Define dynamic loading.

To obtain better memory-space utilization dynamic loading is used. With dynamic loading, a routine is not loaded until it is called. All routines are kept on disk in a reloadable load format. The main program is loaded into memory and executed. If the routine needs another routine, the calling routine

checks whether the routine has been loaded. If not, the reloadable linking loader is called to load the desired program into memory.

16. Define Swapping (APR'15) (APR'16)

A process needs to be in memory to be executed. However a process can be swapped temporarily out of memory to a backing store and then brought back into memory for continued execution. This process is called swapping.

17. Define dynamic linking.

Dynamic linking is similar to dynamic loading, rather than loading being postponed until execution time, linking is postponed. This feature is usually used with system libraries, such as language subroutine libraries. A stub is included in the image for each library routine reference. The stub is a small piece of code that indicates how to locate the appropriate memory-resident library routine, or how to load the library if the routine is not already present.

18. What is External Fragmentation? (APR 2014)

External Fragmentation happens when a dynamic memory allocation algorithm allocates some memory and a small piece is left over that cannot be effectively used. If too much external fragmentation occurs, the amount of usable memory is drastically reduced. Total memory space exists to satisfy a request, but it is not contiguous.

19. What are the common strategies to select a free hole from a set of available holes?

The most common strategies are

- a. First fit
- b. Best fit
- c. Worst fit

20. What do you mean by best fit?(APR 2012)

Best fit allocates the smallest hole that is big enough the entire list has to be searched; unless it is sorted by size this strategy produces the smallest leftover hole.

21. What do you mean by first fit?(APR 2012)

First fit allocates the first hole that is big enough. Searching can either start at the beginning of the set of holes or where the previous first-fit search ended. Searching can be stopped as soon as a free hole that is big enough is found.

22. What are the criteria for LRU page replacement algorithms?(APR 2012)

- LRU stands for least recently used

- LRU replacement associates with each page the time of that page's last use.
- When a page must be replaced LRU chooses a page that has not been used for the longest period of time.
- This strategy is apt for looking backward in time.

23. Define effective access time.

Let p be the probability of a page fault ($0 \leq p \leq 1$). The value of p is expected to be close to 0; that is, there will be only a few page faults. The effective access time is $\text{Effective access time} = (1-p) * ma + p * \text{page fault time}$.
 ma : memory-access time

24. Write the difference between internal and external fragmentation.

Internal Fragmentation is the area in a region or a page that is not used by the job occupying that region or page. This space is unavailable for use by the system until that job is finished and the page or region is released.

25. Explain the difference between paging and segmentation.

	Paging	Segmentation
Length	Fixed length	Variable length
Address space	One dimensional address	Two dimensional address
Protection	Not easy	Good
Sharing	Not easy	Good
Fragment	Internal fragment	External fragment
Linking	Static linking	Dynamic linking
Loading	Dynamic loading	Dynamic loading

26. What are overlays?

To enable a process to be larger than the amount of memory allocated to it, overlays are used. The idea of overlays is to keep in memory only those instructions and data that are needed at a given time. When other instructions are needed, they are loaded into space occupied previously by instructions that are no longer needed.

27. Define paging? (NOV '15) (NOV '18)

It is a memory management scheme. This is another possible solution to the external fragmentation. This allows the physical address space of a process to be non-contiguous thus allowing a process to be allocated in physical memory wherever the free space is available.

28. What is demand paging? (APR 2016) (or)

What is demand paging? What are the major problems to implement demand paging? (APR 2017)

Demand Paging is the Concept in a system that the operating system copies a disk page into physical memory only if an attempt is made to access it and that page is not already in memory.

There are two major problems to implement demand paging. They are frame-allocation and page-replacement. To solve the problem the following steps has to be followed. If we have multiple processes in memory, we must decide how many frames to allocate to each process. Further, when page replacement is required, we must select the frames that are to be replaced. Designing appropriate algorithms to solve these problems is an important task, because disk I/O is so expensive. Even slight improvements in demand-paging methods yield large gains in system performance.

29. What is TLB? What is the advantage of TLB? (Nov' 2017)

TLB stands for Translation Look aside Buffer. In a cached system, the base addresses of the last few referenced pages is maintained in registers called the TLB that aids in faster lookup. TLB contains those page-table entries that have been most recently used. Normally, each virtual memory reference causes 2 physical memory accesses one to fetch appropriate page table entry, and one to fetch the desired data.

Using TLB in between, this is reduced to just one physical memory access in cases of TLB hit. Problem with paging is that, extra memory references to access translation tables can slow programs done by a factor of 2 or 3. Too many entries in the translation tables to keep them all loaded in fast processor memory. The standard solution to this problem is to use a special, small fast lookup hardware cache; called TLB. The TLB is associative high speed memory.

30. Write the three ways to deal the deadlock problem? (Nov' 2018)**Deadlock Prevention.**

Disallow one of the four necessary conditions for deadlock.

Deadlock Avoidance.

Do not grant a resource request if this allocation have the potential to lead to a deadlock.

Deadlock Detection.

Always grant resource request when possible. Periodically check for deadlocks. If a deadlock exists, recover from it.

11 Marks

1. Write short notes on Dead lock and its characteristics?(6 Marks)

A process request for resource if it is not available at the time, a process enters wait state. Waiting process may never again change the state because the resource they requested are held by other process, this situation is called deadlock.

Example:

Suppose a computer has one tape drive and one plotter, the process A and B request tape drive and plotter respectively .Both request are granted.

Now A request the plotter and B request tape drive. Without giving up of its resources, then both the request cannot be granted. This situation is called deadlock.

Example

Semaphores A and B, initialized to 1

```
P0      P1
wait (A); wait(B);
wait (B); wait(A);
```

SYSTEM MODEL:

Under the normal mode of operation a process may utilize a resource in only the following sequence:

1. Request:

If the request cannot be granted immediately(eg. the resource is being used by another process), then the requesting process must wait until it can acquire the resource.

2. Use:

The process can operate on the resource (for eg.if the resource is a printer, the process can print)

3. Release:

The process releases the resource.

Deadlock Characterization

Necessary condition:

A deadlock situation can arise if the following four conditions hold simultaneously in a system:

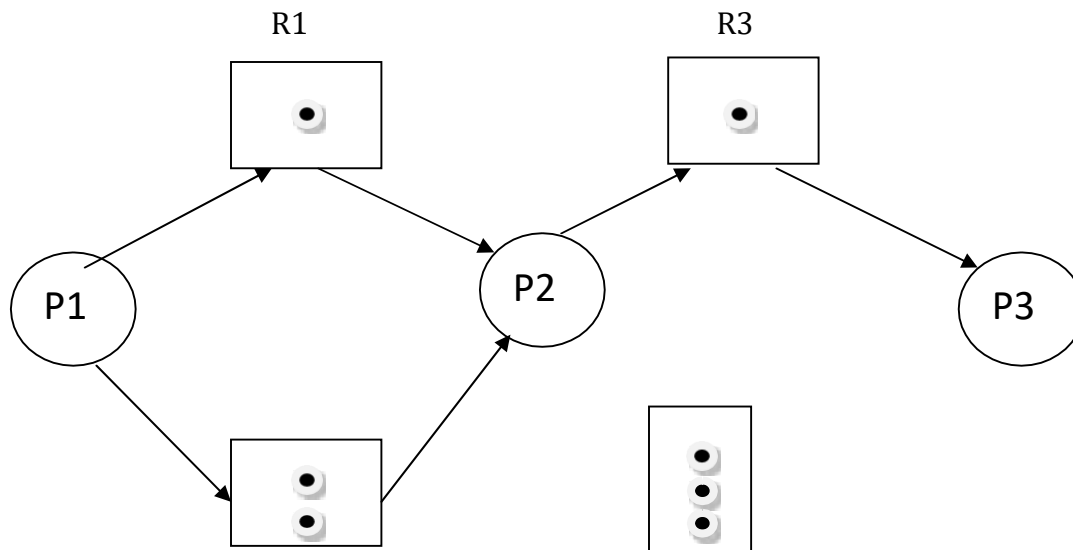
- Mutual exclusion: At least one resource must be held in a non-sharable mode; that is, only one process at a time can use a resource. If another process requests that resource, the requesting process must be delayed until the resource has been released.
- Hold and wait: a process must be holding at least one resource is waiting to acquire additional resources held by other processes.
- No preemption: a resource can be released only voluntarily by the process holding it, after that process has completed its task.
- Circular wait: there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_0 is waiting for a resource that is held by P_0 .
- Resource allocation graph: deadlocks can be described more precisely in terms of directed graph called a system resource allocation graph.

This graph consists of a set of vertices V and a set of edges E .

-the set of vertices V is partitioned into two different types;

P-set containing all active processes.

R-set consisting of all resource types.



Request edge: Directed edge $p_i \rightarrow r_j$ is called a request edge.

Assignment edge: A directed edge $r_j \rightarrow p_i$ is called an assignment edge.

2. Explain Deadlock Prevention in detail? (6 Marks) (APR'15)

Deadlock prevention:

-Deadlock occurs if each of the four necessary conditions hold.

-By ensuring that at least one of these conditions cannot hold; the occurrence of deadlock can be prevented.

- **Mutual Exclusion** – not required for sharable resources; must hold for non sharable resources.
- **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources.

Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none. Low resource utilization; starvation possible.

- **No Preemption** – If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released.

Preempted resources are added to the list of resources for which the process is waiting.

Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting.

- **Circular Wait** – impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration.

3. Explain Deadlock Avoidance in details? (11 Marks) (APR 2012)

Simplest and most useful model requires that each process declare the maximum number of resources of each type that it may need.

The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition.

Resource-allocation state is defined by the number of available and allocated resources, and the maximum demands of the processes.

Safe State:

When a process requests an available resource, system must decide if immediate allocation leaves the system in a safe state.

System is in safe state if there exists a safe sequence of all processes.

Sequence $\langle P_1, P_2, \dots, P_n \rangle$ is safe if for each P_i , the resources that P_i can still request can be satisfied by currently available resources + resources held by all the P_j , with $j < i$.

If P_i resource needs are not immediately available, then P_i can wait until all P_j have finished. When P_j is finished, P_i can obtain needed resources, execute, return allocated resources, and terminate.

When P_i terminates, P_{i+1} can obtain its needed resources, and so on.

Basic Facts:

If a system is in safe state then no deadlocks. If a system is in unsafe state then possibility of deadlock to happen.

Avoidance and ensure that a system will never enter an unsafe state.

Safe, unsafe, dead lock state



Resource-Allocation Graph Algorithm

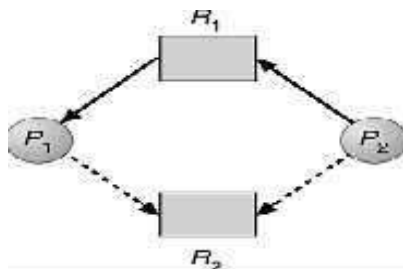
- Claim edge P_i

$P_i \rightarrow R_j$

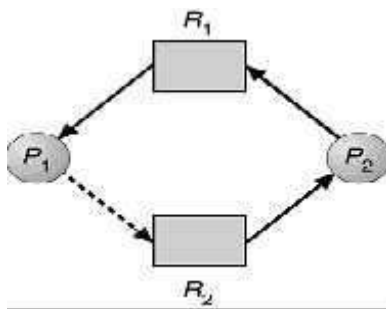
Indicated that process P_j may request resource R_j ; represented by a dashed line.

- Claim edge converts to request edge when a process requests a resource.
- When a resource is released by a process, assignment edge reconverts to a claim edge.
- Resources must be claimed a priori in the system.

Resource allocation graph for dead lock avoidance (for one instance of each resource)



Unsafe state in resource allocation graph



4. Explain Banker's Algorithm (6) (APR 2012, NOV '15)

- Applicable to systems with multiple instances of each resource type.
- Each process must declare the maximum number of instance required for each resource type upon entering the system.
- When a process requests a set of resources, the system determines whether the allocation of these resources will have the system in a safe state.
- Yes: allocate the resources
- No: the process must wait

Data Structures for the Banker's Algorithm

- **Available:** Vector of length m. If available [j] = k, there are k instances of resource type R_j available.
- **Max:** $n \times m$ matrix. If Max [i,j] = k, then process P_i may request at most k instances of resource type R_j .
- **Allocation:** $n \times m$ matrix. If Allocation [i,j] = k, then P_i is currently allocated k instances of R_j .
- **Need:** $n \times m$ matrix. If Need [i,j] = k, then P_i may need k more instances of R_j to complete its task.

$$\text{Need [i,j]} = \text{Max[i,j]} - \text{Allocation [i,j]}.$$

Safety Algorithm

1. Let Work and Finish be vectors of length m and n, respectively.

Initialize Work = Available

Finish [i] = false for $i = 1, 2, 3, \dots, n$.

2. Find an i such that both:

(a) Finish [i] = false

(b) $\text{Need}_i \leq \text{Work}$

If no such i exists, go to step 4.

3. $\text{Work} = \text{Work} + \text{Allocation}_i$

Finish[i] = true go to step 2.

4. If Finish [i] == true for all i, then the system is in a safe state.

Resource-Request Algorithm for Process P_i

Request_i = request vector for process P_i .

If Request [j] = k then process P_i wants k instances of resource type R_j .

1. If Request_i <= Need_i go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim.

2. If Request_i <= Available, go to step 3. Otherwise P_i must wait, since resources are not available.

3. Pretend to allocate requested resources to P_i by modifying the state as follows:

Available = Available – Request_i;

Allocation_i = Allocation_i + Request_i;

Need_i = Need_i – Request_i;

- If safe the resources are allocated to P_i .
- If unsafe P_i must wait, and the old resource-allocation state is restored

5. Explain Deadlock Detection (8 Marks)? (NOV 2011) (NOV 2012)

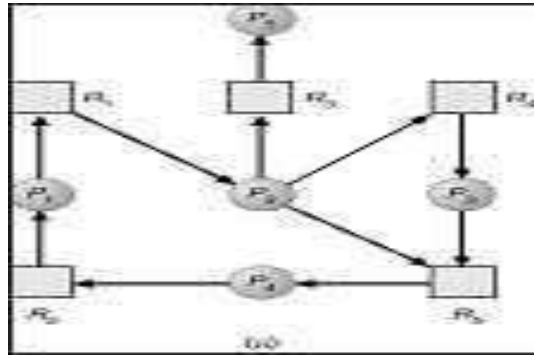
In an Deadlock detection the following methods are followed

- Allow system to enter deadlock state
- Detection algorithm
- Recovery scheme

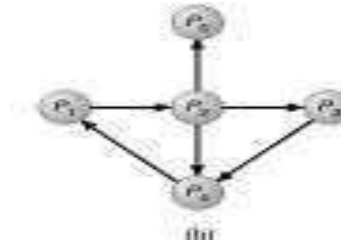
Single Instance of Each Resource Type

- Maintain wait-for graph Nodes are processes.
- $P_i < P_j$ if P_i is waiting for P_j .
- Periodically invoke an algorithm that searches for a cycle in the graph.
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph.

Several Instances of a Resource Type Resource allocation graph



Corresponding wait graph



- Available: A vector of length m indicates the number of available resources of each type.
- Allocation: An $n \times m$ matrix defines the number of resources of each type currently allocated to each process.
- Request: An $n \times m$ matrix indicates the current request of each process.
- If $\text{Request}[i, j] = k$, then process P_i is requesting k more instances of resource type R_j .

Detection Algorithm

1. Let Work and Finish be vectors of length m and n , respectively. Initialize as follows:

(a) $\text{Work} = \text{Available}$

(b) For $i = 1, 2, \dots, n$,

If $\text{Allocation}[i] \neq 0$, then $\text{Finish}[i] = \text{false}$;

Otherwise, $\text{Finish}[i] = \text{true}$.

2. Find an index i such that both:

(a) $\text{Finish}[i] == \text{false}$

(b) $\text{Request}_i \leq \text{Work}$

If no such i exists, go to step 4.

3. $\text{Work} = \text{Work} + \text{Allocation}_i$

$\text{Finish}[i] = \text{true}$

go to step 2.

4. If $\text{Finish}[i] == \text{false}$, for some i , $1 \leq i \leq n$, then the system is in deadlock state. Moreover,

If $\text{Finish}[i] == \text{false}$, then P_i is deadlocked.

Detection-Algorithm Usage

- When, and how often, to invoke depends on.

- How often a deadlock is likely to occur.
- How many processes will need to be rolled back.

One for each disjoint cycle

- If detection algorithm is invoked arbitrarily, there may be many cycles in the resource graph and so we would not be able to tell which of the many deadlocked processes “caused” the deadlock.

6. Write short notes on Recovery from Deadlock (5 Marks) (NOV 2011) (NOV 2012)

Process Termination

- Abort all deadlocked processes.
- Abort one process at a time until the deadlock cycle is eliminated.
- In which order should we choose to abort.
- Priority of the process.
- How long process has computed, and how much longer to completion.
- Resources the process has used.
- Resources process needs to complete.
- How many processes will need to be terminated.
- Is process interactive or batch system

Recovery from Deadlock:

Resource Preemption

- Selecting a victim – minimize cost.
- Rollback – return to some safe state, restart process for that state.
- Starvation – same process may always be picked as victim, include number of rollback in cost factor.

7. Explain about contiguous memory allocation (5Marks) (APR 2014)

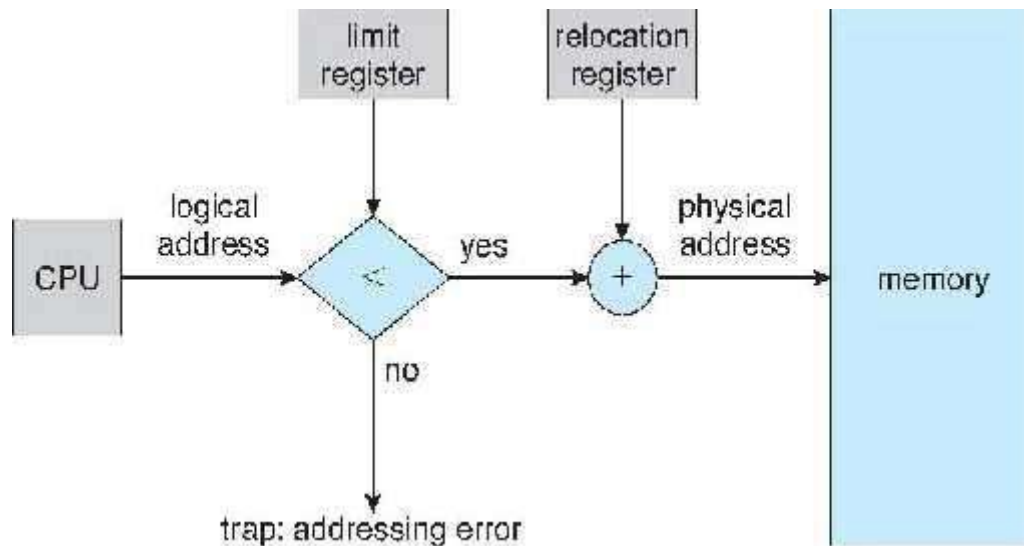
Contiguous Allocation

Main memory usually divided into two partitions: Resident operating system usually held in low memory with interrupt vector. User processes held in high memory.

Single-partition allocation

In the below figure the Relocation register scheme used to protect user processes from each other, and from changing operating system code and data. Relocation register contains value of smallest physical address, limit register contains range of logical addresses each logical address must be less than the limit register. MMU contains logical address dynamically.

Hardware support for relocation and limit register



Multiple partition allocation

In this method we can store several processes in memory at the same time before allocate the memory to variable processes they are waiting in ready queue one of the simplest methods for memory allocation is to divide the memory into a number of fixed size partition.

Each process may contain only one process when a partition is free; a process is selected from the input queue and is loaded into the free partition. When the process terminates the partition becomes available for another process. The os keeps a table indicating which parts of memory are available and which are occupied a free block of memory is called a hole.

When a process needs to be in the memory, a hole large enough to satisfy this request is found and is allocated to that process the rest is kept for future request.

The set of holes is searched first to determine which hole to allocate first fit, best fit and worst fit are the most common strategies used to select the free hole from the set of available holes.

First-fit: Allocate the *first* hole that is big enough.

Best-fit: Allocate the *smallest* hole that is big enough; must search entire list, unless ordered by size. Produces the smallest left over hole.

Worst-fit: Allocate the *largest* hole; must also search entire list. Produces the largest leftover hole.

Fragmentation:

-memory fragmentation can be internal as well as external

External fragmentation:

-External fragmentation exists when enough total memory space exists to satisfy a request; but it is not contiguous, storage is fragmented into large number of small holes.

-This fragmentation problem can be severe. There may be a block of free memory between every two processes.

Internal fragmentation:

- The memory allocated to a process may be slightly larger than the requested memory.
- The difference between these two numbers is internal fragmentation-memory that is internal to partition but is not being used.

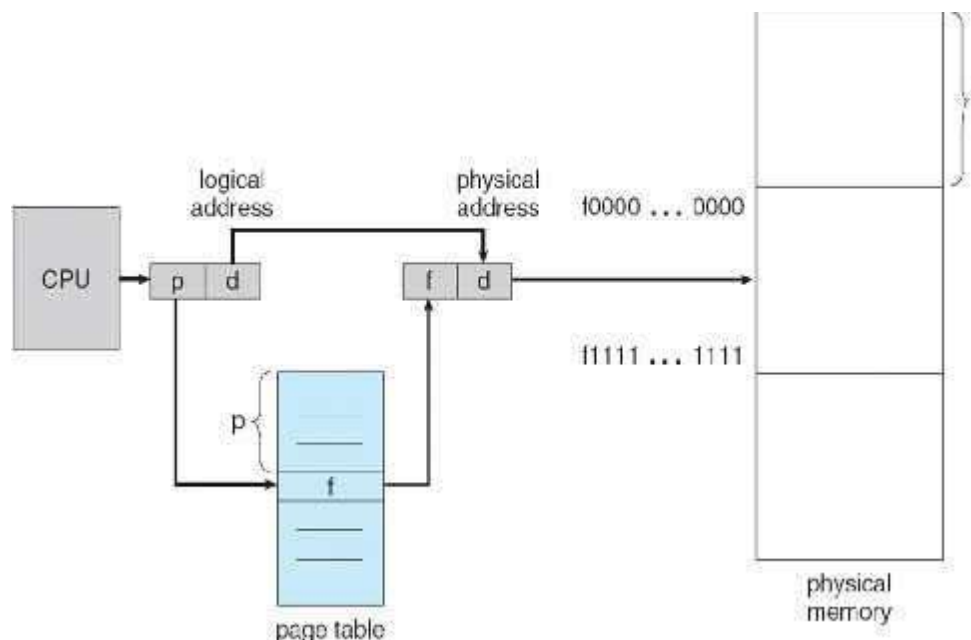
8. Explain Paging in detail? (6) (APR 2014, NOV '15)

It is a memory management scheme .This is another possible solution to the external fragmentation. This allow the physical address space of a process to be non-contiguous thus allowing a process to be allocated in physical memory where ever the free space is available.

Basic method:-

- Physical memory is broken into fixed size blocks called frames.
- Similarly the logical memory is broken into block of same size called pages.
- When a process is to be executed its pages are loaded into available memory frames from the backing store.
- The backing store is divided into fixed size blocks that are of the same size as memory frames.

Paging hardware algorithm:

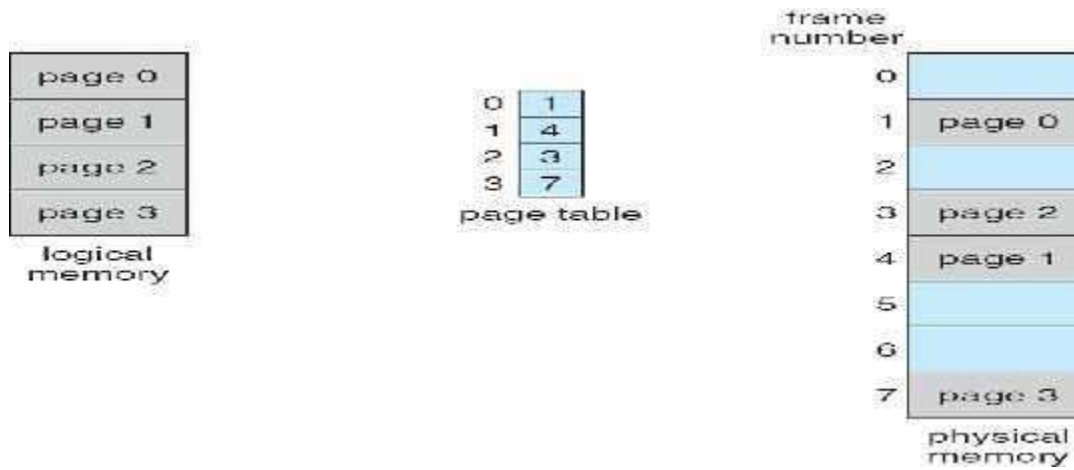


Every address generated by the CPU is divided into two parts: page number and page offset.

- The page number is used to index into a page table.
- The page table contains the base address of each page in physical memory.
- The base address is combined with the page offset to define the physical address that is sent to the memory unit.
- The page size is defined by the hardware and is usually power of 2 varying between 512bytes and 8192bytes per page.

Total logical address space 2^m and Page size is 2^n , When address is created and $m-n$ bits = page number (p) then n bits = displacement (d) p acts as an index to page table.

Paging example



Here the page size is 4 bytes and the size of the physical memory is 32 bytes (8 pages). Logical address zero maps to page 0 offset 0, indexing into page table. We find that page 0 is in frame 1, thus the logical address maps to physical address 4. Logical address 3 (page 0, offset 3) maps to physical address 7, logical address 4 is (page 1, offset 0) according to the page table, page 1 is mapped to frame 6. Thus logical address 4 maps to physical address 28.

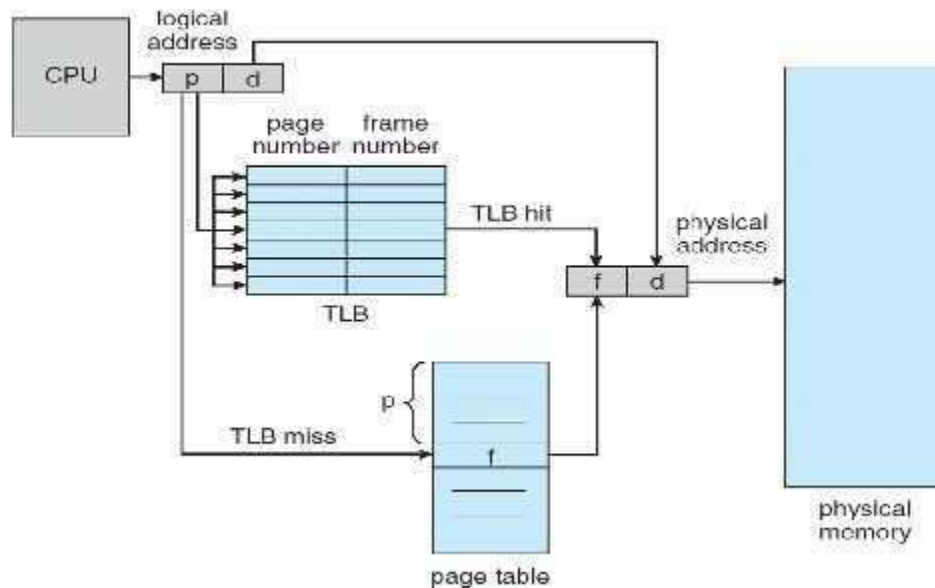
Notice that paging itself is a form of dynamic relocation. Every logical address is bound by paging hardware to some physical address. When we use a paging scheme we have no external fragmentation. We may have some internal fragmentation.

9. Explain TLB (6 Marks)

TLB stands for Translation Look aside Buffer. In a cached system, the base addresses of the last few referenced pages are maintained in registers called the TLB that aids in faster lookup. TLB contains those page-table entries that have been most recently used. Normally, each virtual memory reference causes 2 physical memory accesses: one to fetch appropriate page table entry, and one to fetch the desired data.

Using TLB in between, this is reduced to just one physical memory access in cases of TLB hit. Problem with paging is that, extra memory references to access translation tables can slow programs down by a factor of 2 or 3. Too many entries in the translation tables to keep them all loaded in fast processor memory. The standard solution to this problem is to use a special, small fast lookup hardware cache; called TLB. The TLB is associative high speed memory.

Paging hardware with TLB



A translation buffer is used to store a few of the translation table entries. It is very fast but only for a small numbers of entries.

On each memory reference has

1. First ask TLB it knows about the page the page, if so the reference proceeds fast.
2. If TLB has no information for page must go through page and segment table to get information. Reference takes a long time, but gives the information for this page to TLB so it will know it for next reference.

TLB hit: If the page member is found in TLB; TLB hit

TLB miss: If the page number is not found in TLB; TLB miss

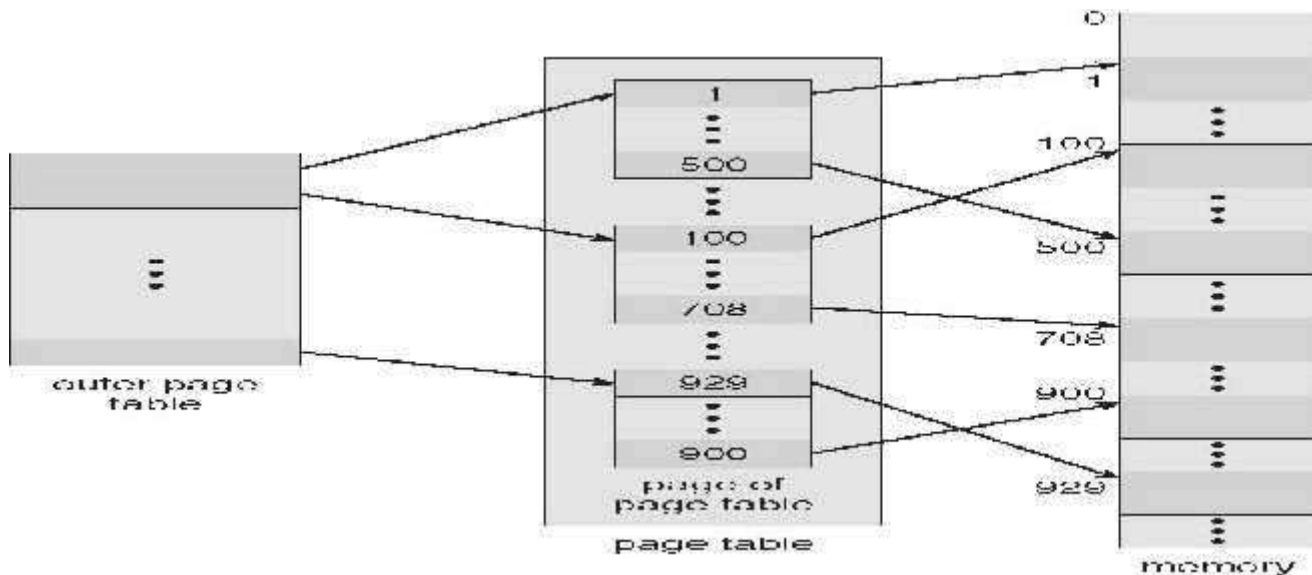
10. Describe the paging memory management scheme in details? (APR 2012)

Structure of the Page Table:

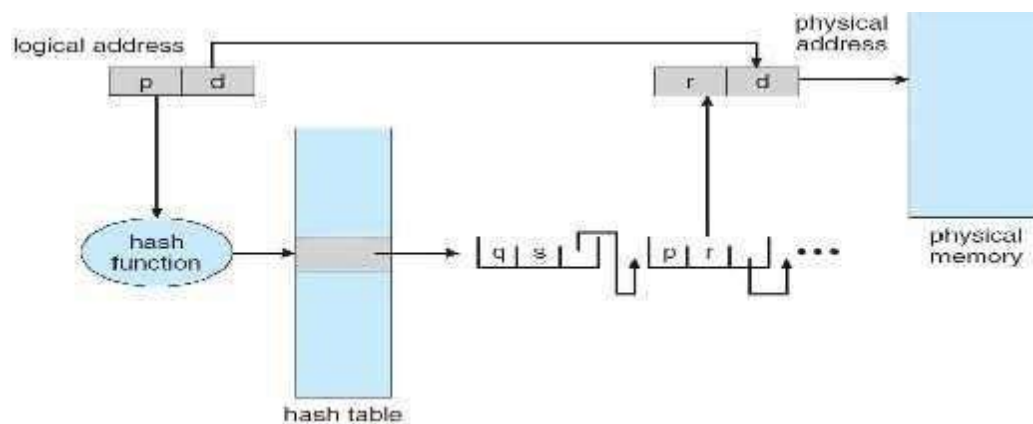
- Hierarchical Paging
- Hashed Page Tables
- Inverted Page Tables

Hierarchical Page Tables:

Recent computer system supports a large logical address space from 2^{32} to 2^{64} . In this system the page table becomes large. It is difficult to allocate contiguous main memory for page table. To solves this problem; two level page table scheme is used. The page table is divided into number of smaller pieces.



Hashed Page Tables



Hashed Page Tables handles the address space larger than 32 bits. The virtual page number is used as hashed value. Linked list is used in the hash table. Each entry contains the linked list of element that hash to the same location.

Each element in the hash table contains the following fields

1. Virtual page number
2. Mapped page frame value
3. Pointer to the next element in the linked list

Working methods:

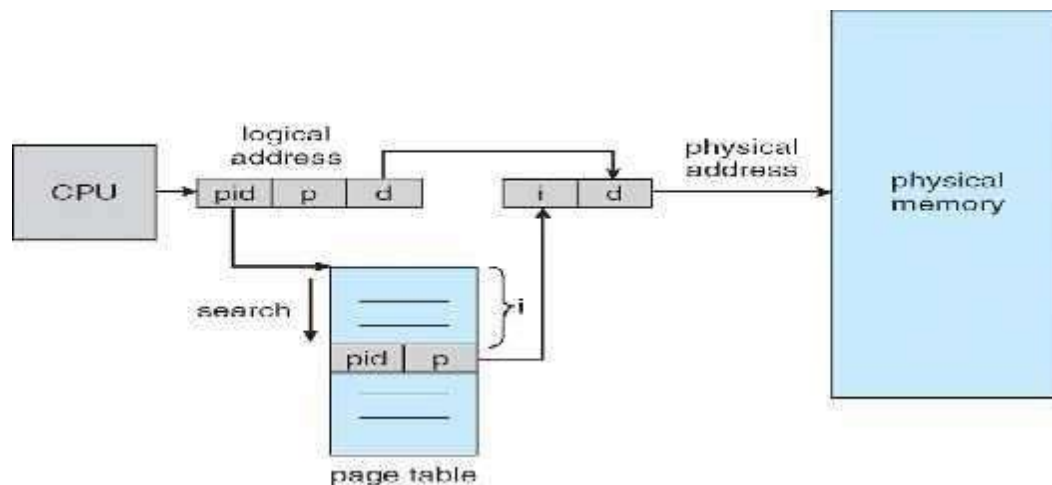
1. Virtual page number is taken from virtual address
2. Virtual page number is hashed with the hash table
3. Virtual page number is compared with the first element of linked list
4. Both the value is matched, that value (i.e. page frame) used for calculating physical address.
5. If both values are not matched the entire linked list is searched for a matching.

Inverted Page Table:

An address space have grown to 64 bits the size of traditional page table becomes a problem even with 2 level page tables the tables themselves can become too large. Inverted Page Table is used to solve this problem. The inverted Page Table has one entry for each real page of memory. A physical page table instead of a logical one. The physical page table is often calls as Inverted Page Table.

This table contains one entry per age frame .An Inverted Page Table is very good at mapping from physical page to logical page number, but not very good at mapping from virtual page number to physical page number.

There is no other hardware or registers dedicated to memory mapping the TLB can be quite larger, so that missing entry faults are rare. With an Inverted Page Table, most address translations are handled by the TLB when there is a miss in the TLB, the OS is notified and TLB miss handler is invoked. Hashing is used to speed up the search.



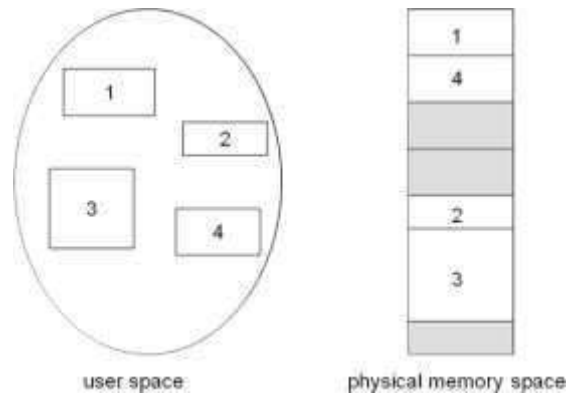
11. Discuss in Segmentation in details? (11) (NOV 13) (NOV 18)

Segmentation

Segmentation is a Memory Management technique in which memory is divided into variable sized chunks which can be allocated to processes. Each chunk is called a segment. A table stores the information about all such segments and is called **Global Descriptor Table (GDT)**. A GDT entry is called Global Descriptor. It comprise of: Logical Address to Linear Address.

A program is a collection of segments. A segment is a logical unit such as: main program, procedure, function, method, object, local variables, global variables, common block, stack, symbol table, arrays.

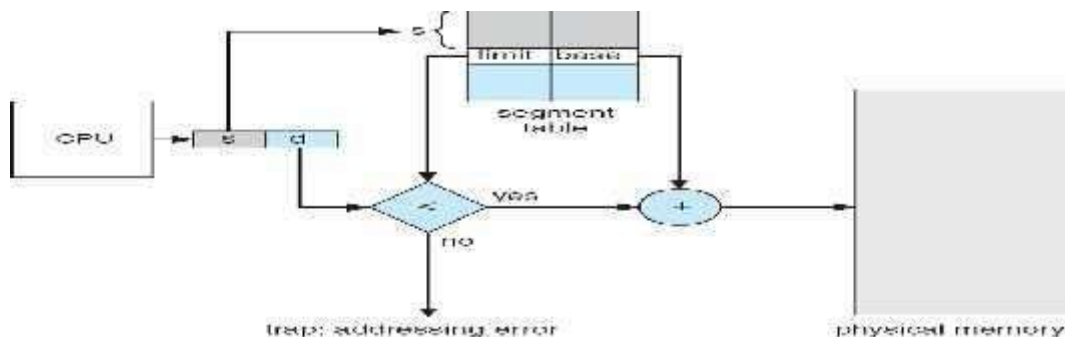
Logical View of Segmentation



- Logical address consists of a two tuple. <segment-number, offset>
- **Segment table** – maps two-dimensional physical addresses; each table entry has:
 - base** – contains the starting physical address where the segments reside in memory.
 - limit** – specifies the length of the segment
- **Segment-table base register (STBR)** points to the segment table's location in memory.
- **Segment-table length register (STLR)** indicates number of segments used by a program.

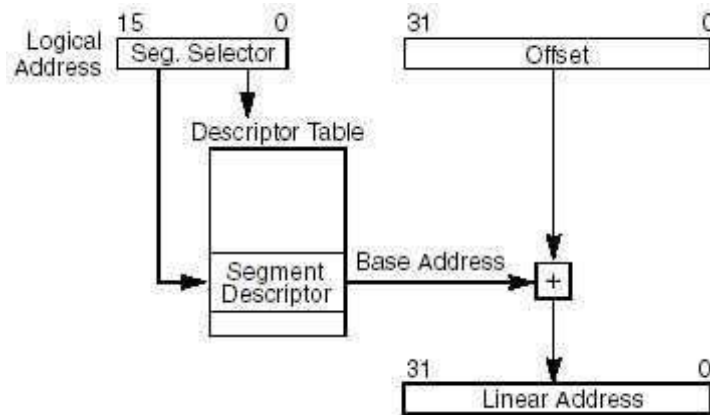
Segment number s is legal if $s < \text{STLR}$

Segmentation Hardware



To translate a logical address into a linear address, the processor does the following:

1. Uses the offset in the segment selector to locate the segment descriptor for the segment in the GDT or LDT and reads it into the processor. (This step is needed only when a new segment selector is loaded into a segment register.)
2. Examines the segment descriptor to check the access rights and range of the segment to insure that the segment is accessible and that the offset is within the limits of the segment.
3. Adds the base address of the segment from the segment descriptor to the offset to form a linear address.



If paging is not used, the processor maps the linear address directly to a physical address (that is, the linear address goes out on the processor's address bus). If the linear address space is paged, a second level of address translation is used to translate the linear address into a physical address.

Segment Selector

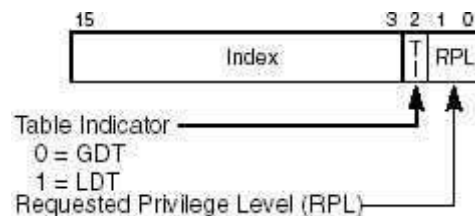
A segment selector is a 16-bit identifier for a segment. It does not point directly to the segment, but instead points to the segment descriptor that defines the segment. A segment selector contains the following items:

Index

Index bits 3 through 15. Selects one of 8192 descriptors in the GDT or LDT. The processor multiplies the index value by 8 the number of bytes in a segment descriptor and adds the result to the base address of the GDT or LDT from the GDTR or LDTR register, respectively.

TI (table indicator) flag

In table indication flag is Bit 2. Specifies the descriptor table to use: clearing this flag selects the GDT and setting this flag selects the current LDT.



Segmented Paging

Benefits: faster process start times, faster process growth, memory sharing between processes.

Costs: somewhat slower context switches, slower address translation.

- Pure paging system => (virtual address space)/ (page size) entries in page table.

12. Consider the following snapshot of a system: (11) (NOV '13)

Allocation	Max	Available
------------	-----	-----------

	ABCD	ABCD	ABCD
Po	0012	0012	1520
P1	1000	1750	
P2	1354	2356	
P3	0632	0652	
P4	0014	0656	

- What is the content of the matrix Need?
- Is the system in a safe state?
- If a request from process P1 arrives for (0, 4, 2, 0), can the request be granted immediately?

A. the content of the matrix Need

	A	B	C	D
P0	0	0	0	0
P1	0	7	5	0
P2	0	0	2	0
P3	0	2	0	0
P4	0	6	4	2

B. the system in a safe state

P0 P2 P3 P4 P1

C.request from process P1 arrives for (0,4,2,0), can the request be granted immediately

Yes request from process P1 arrives for (0, 4, 2, 0), can the request be granted immediately .

13. Consider the following snapshot of a system: (11) (NOV 14)

	Allocation			Max		
	A	B	C	A	B	C
Po	0	0	2	2	0	3
P1	1	1	0	2	3	5
P2	0	0	1	1	2	3
P3	1	0	0	2	0	3
P4	0	0	2	0	1	5

Available :

A=0. B=2 c=1

The content of the matrix Need

	A	B	C
P0	2	0	1
P1	1	2	5
P2	1	2	2

P3	1	0	3
P4	0	1	3

The system is in unsafe state because the requirement of all the process are more than the available resources

14. What are the various address translation mechanism used in paging (11) (APR 16)

Address Translation

Memory Management Unit (MMU) translates virtual address into physical address for each load and store. Software (privileged) controls the translation. CPU view only the Virtual addresses. Each process has its own memory space [0, high] know as Address space. Memory or I/O device view the Physical addresses.

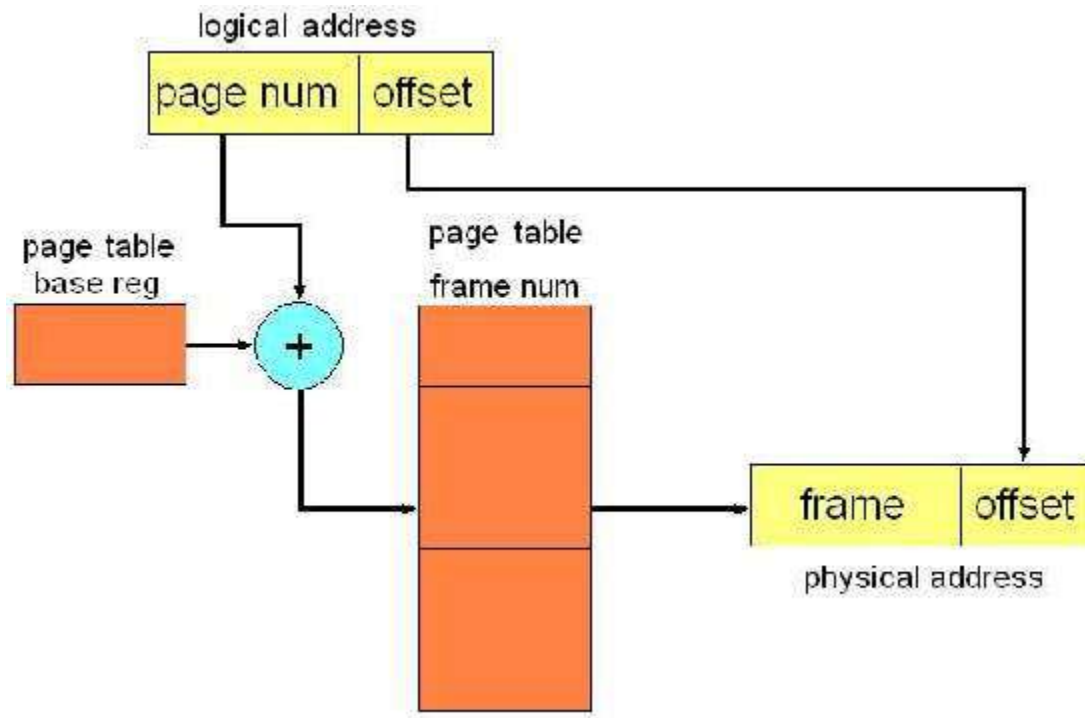
The basic model:



Paging

Address translation by paging is similar to segmentation. A virtual address is comprised of a page number and an offset. Page tables are used to translate the page number into a *page frame* number; that is, the physical location of the page in main memory. The main difference between pages and segments is that whereas segments have varying length determined by the structure of the user program, pages have a uniform length determined by the hardware architecture.

- Typical page size is around 4K bytes
- Pages are transparent to the user or programmer
- Pages are independent of the logical program structure



example Suppose we have a logical and physical address size of 24 bits, with a page size of 2K. How many bits are there in a page number? in an offset? How many entries are there in a page table?

In general, if a logical address is n bits and the page size is 2^m , we have

- the logical address space is 2^n bytes
- the logical address space contains 2^{n-m} pages
- page number is $n-m$ bits, offset is m bits
- the page table contains 2^{n-m} entries

Paging greatly simplifies the management of physical memory by the operating system. Memory can be viewed as an array of pages. It can be managed using a frame table; each frame is either free, or allocated to some address space. Paging eliminates the external fragmentation problem, but is still subject to internal fragmentation.

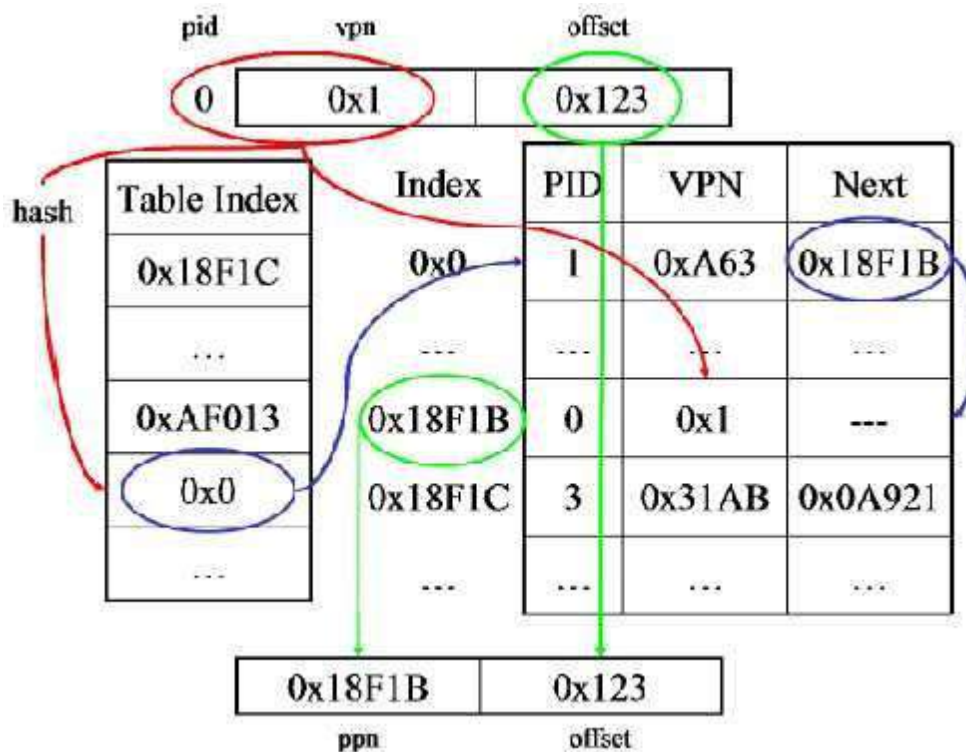
Both of these address translation techniques are examples of *direct mapping*, so named because the translation is based on direct indexing into the segment or page table.

Alternatives to Direct Mapping

Translation Lookaside Buffer (TLB) - A fast cache memory used to hold a subset of the (page number) --> (frame number) mapping information

Hashed Page Tables - Use a hash table to map page numbers to frame numbers

Inverted Page Tables - One entry per frame. The entry contains a (pid, virtual page number) pair. The table can be searched using hashing, as shown below:



15. Consider the following snapshot of a system: (11) (APR 16)

	Allocation	Max	Available
	ABCD	ABCD	ABCD
Po	0011	0011	1522
P1	1001	1751	
P2	1351	2352	
P3	0531	1652	
P4	0011	5651	

Answer the following using banker's algorithm

- What is the content of the matrix Need?
- Is the system in a safe state?

a. the content of the matrix Need

	A	B	C	D
P0	0	0	0	0
P1	0	7	5	0
P2	1	0	0	1
P3	1	1	2	1
P4	5	6	4	0

b. the system in a safe state

No the system is not in the safe state

16. (a) In what sequence the process may utilize the resource in a normal mode of operation. (2) (APR '17)

Under the normal mode of operation, a process may utilize a resource in only the following sequence:

- **Request** If the request cannot be granted immediately (for example, if the resource is a printer, the process can acquire the resource.
- **Use** The process can operate on the resource (for, example, if the resource is a printer, the process can print on the printer).
- **Release** The process releases the resource.

(b) Consider the following snapshot of a system: (9) (APR '17)

	Allocation	Max	Available
	ABCD	ABCD	ABCD
Po	0012	0012	1520
P1	1000	1750	
P2	1354	2356	
P3	0632	0652	
P4	0014	0656	

- What is the content of the matrix Need?
- Is the system in a safe state?
- If a request from process P1 arrives for (0, 4, 2, 0), can the request be granted immediately?
(Refer question Number 12, Page number 22)

17. (a) Why are segmentation and paging sometimes combined into one scheme? (2) (APR '17)

Segmentation and paging are often combined in order to improve upon each other. Segmented paging is helpful when the page table becomes very large. By paging the segments, we reduce wasted memory due to external fragmentation as well as simplify the allocation.

(b) Consider a paging system with the page table stored in memory. (5) (APR '17)

- If a memory reference takes 200 nanoseconds, how long does a paged memory reference take?

Answer: 400 nanoseconds. 200 ns to access the page table plus 200 ns to access the word in memory.

(ii) If we add TLBs, and 75 percent of all page-table references are found in the TLBs, what is the effective memory reference time? (Assume that finding a page-table entry in the TLBs take zero time, if the entry is there.)

Answer: 250 nanoseconds. 75% of the time it's 200 ns, and the other 25% of the time it's 400ns, so the equation is: Effective access time= $0.75 * (200 \text{ nanoseconds}) + 0.25 * (400 \text{ nanoseconds}) = 250 \text{ nanoseconds}$

(c) Consider the following segment table (4) (APR '17)

Segment	Base	Length
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

What are the physical addresses for the following logical addresses?

(i) 0,430

(ii) 1,10

(iii) 2,500

(iv) 3,400

(v) 4,112.

Answer:

(i) $219 + 430 = 649$

(ii) $2300 + 10 = 2310$

(iii) illegal reference, trap to operating system

(iv) $1327 + 400 = 1727$

(v) illegal reference, trap to operating system

18. Consider the following snapshot of the system. (11) (Nov '16)

	Allocation	Max	Available
	ABC	ABC	ABC
Po	010	000	000
P1	200	202	
P2	303	000	
P3	211	100	

P4 002 002

- a. Is the system in a safe state?
- b. What will happen that the process p2 makes one additional request for an instance of type C?

Answer:

- a. Yes, the system is in safe state as its sequence would be P₀, P₂, P₃, P₄, P₁.
- b.

Allocation	Max	Available
ABC	ABC	ABC
P ₀ 010	000	000
P ₁ 200	202	
P ₂ 303	001	
P ₃ 211	100	
P ₄ 002	002	

The system will be in an unsafe state if process p₂ makes one additional request for an instance of type C, because the requirement of all the processes are more than the available resources.

19. (a) Consider the following segment table (4) (Nov '16)

Segment	Base	Length
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

What are the physical addresses for the following logical addresses?

- (i) 0,430
- (ii) 1,10
- (iii) 2,500

Answer:

- (i) $219 + 430 = 649$
- (ii) $2300 + 10 = 2310$
- (iii) illegal reference, trap to operating system

(b) Compare the main memory organization schemes of contiguous memory allocation, pure segmentation, and pure paging with respect to the following issues: (7) (Nov '16)

(i) External fragmentation

(ii) Internal fragmentation

(iii) Ability to share code across processes

- Contiguous memory allocation scheme suffers from external fragmentation as address spaces are allocated contiguously and holes develop as old processes die and new processes are initiated.
- It also does not allow processes to share code, since a process's virtual memory segment is not broken into non-contiguous fine grained segments. Pure segmentation also suffers from external fragmentation as a segment of a process is laid out contiguously in physical memory and fragmentation would occur as segments of dead processes are replaced by segments of new processes.
- Segmentation, however, enables processes to share code;
- for instance, two different processes could share a code segment but have distinct data segments. Pure paging does not suffer from external fragmentation, but instead suffers from internal fragmentation. Processes are allocated in page granularity and if a page is not completely utilized, it results in internal fragmentation and a corresponding wastage of space.
- Paging also enables processes to share code at the granularity of pages.

Pondicherry University Questions

2 Marks

1. Define deadlock (UQ NOV'13) (Ref.Pg.No.1 Qn.No.1)
2. Why the page sizes is always power of 2 in memory? (UQ NOV'13) (Ref.Pg.No.3 Qn.No.13)
3. What is a resource allocation graph?(UQ APR'12, APR'15, APR'17) (Ref.Pg.No.4 Qn.No.18)
4. What is External Fragmentation? (APR 2014) (Ref.Pg.No.4 Qn.No.19)
5. What do you mean by best fit?(APR'12) (Ref.Pg.No.4 Qn.No.21)
6. What do you mean by first fit?(APR'12) (Ref.Pg.No.4 Qn.No.22)
7. What are the criteria for LRU page replacement algorithms?(APR 2012) (Ref.Pg.No.5 Qn.No.23)
8. Define Swapping (APR'15, APR'16) (Ref.Pg.No.3 Qn.No.16)

9. What are conditions under which a deadlock situation may arise? Or Write four general strategies for dealing with deadlocks? (APR'15)(NOV'14) (Ref.Pg.No.1 Qn.No.3)
10. Define paging. (NOV '15, NOV '18) (Ref.Pg.No.5 Qn.No.28)
11. Write down three ways to deal with deadlock problem. (NOV '17) (Ref.Pg.No.2 Qn.No.6)
12. What is demand paging? (APR '16, APR '17, NOV'18) (Ref.Pg.No.6 Qn.No.28)
13. What is TLB? What is the advantage of TLB? (APR'17) (Ref.Pg.No.6 Qn.No.28)
14. Write the three ways to deal the deadlock problem? (Nov' 2018) (Ref.Pg.No.6 Qn.No.28)

11 Marks

1. Explain Deadlock Avoidance in details? (UQ APR'12) (Ref.Pg.No.8 Qn.No.3)
2. Describe bankers algorithms?(UQ APR'12, NOV '15) (Ref.Pg.No.9 Qn.No.4)
3. Explain Deadlock Detection? (UQ NOV'11& NOV'12) (Ref.Pg.No.11 Qn.No.5)
4. Write short notes on Recovery from Deadlock? (UQ NOV '11& NOV 12) (Ref.Pg.No.12 Qn.No.6)
5. Explain about contiguous memory allocation? (UQ APR'14) (Ref.Pg.No.13 Qn.No.7)
6. Explain Paging in detail? (UQ APR'14, NOV '15) (Ref.Pg.No.14 Qn.No.8)
7. Describe the paging memory management scheme in details?(UQ APR'12) (Ref.Pg.No.15 Qn.No.10)
8. Discuss in Segmentation in details?(UQ NOV'13, NOV'18) (Ref.Pg.No.19 Qn.No.11)
9. Consider the following snapshot of a system: (UQ NOV'13) (Ref.Pg.No.21 Qn.No.12)

	Allocation	Max	Available
	A B C D	A B C D	A B C D
Po	0 0 1 2	0 0 1 2	1 5 2 0
P1	1 0 0 0	1 7 5 0	
P2	1 3 5 4	2 3 5 6	
P3	0 6 3 2	0 6 5 2	
P4	0 0 1 4	0 6 5 6	

- a. What is the content of the matrix Need?
- b. Is the system in a safe state?
- c. If a request from process P1 arrives for (0, 4, 2, 0), can the request be granted immediately?
10. Explain storage management .A system has 2 A resources 3 B and 6 C resources .5 processes their current allocation and their maximum allocation are shown below. Is the system in a safe state? If so ,show one sequence of processes which allow the system to complete .If not, explain why. (NOV'14) (Ref.Pg.No.22 Qn.No.13)

Allocation			Max		
A	B	C	A	B	C

Po	0	0	2	2	0	3
P1	1	1	0	2	3	5
P2	0	0	1	1	2	3
P3	1	0	0	2	0	3
P4	0	0	2	0	1	5

11. Explain Deadlock Prevention in detail? (APR'15) (Ref.Pg.No.7 Qn.No.2)

12. What are the various address translation mechanism used in paging (APR 16) (Ref.Pg.No.23 Qn.No.14)

13. Consider the following snapshot of a system: (APR 16) (Ref.Pg.No.26 Qn.No.15)

	Allocation	Max	Available
	ABCD	ABCD	ABCD
Po	0011	0011	1522
P1	1001	1751	
P2	1351	2352	
P3	0531	1652	
P4	0011	5651	

Answer the following using banker's algorithm

a. What is the content of the matrix Need?

b. Is the system in a safe state?

14. (a) In what sequence the process may utilize the resource in a normal mode of operation. (2) (APR '17) (Ref.Pg.No.26 Qn.No.16)

(b) Consider the following snapshot of a system: (9) (APR '17)

	Allocation	Max	Available
	ABCD	ABCD	ABCD
Po	0012	0012	1520
P1	1000	1750	
P2	1354	2356	
P3	0632	0652	
P4	0014	0656	

15. (a) Why are segmentation and paging sometimes combined into one scheme? (2) (APR '17)

(b) Consider a paging system with the page table stored in memory. (5) (APR '17) (Ref.Pg.No.27 Qn.No.17)

(i) If a memory reference takes 200 nanoseconds, how long does a paged memory reference take?

(ii) If we add TLBs, and 75 percent of all page-table references are found in the TLBs, what is the effective memory reference time? (Assume that finding a page-table entry in the TLBs take zero time, if the entry is there.)

(c) Consider the following segment table (4) (APR '17)

Segment	Base	Length
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

What are the physical addresses for the following logical addresses?

(iv) 0,430

(v) 1,10

(vi) 2,500

(vii) 3,400

(viii) 4,112.

16. Consider the following snapshot of the system. (Nov '16) (Ref.Pg.No.28 Qn.No.18)

	Allocation	Max	Available
	ABC	ABC	ABC
Po	010	000	000
P1	200	202	
P2	303	000	
P3	211	100	
P4	002	002	

- Is the system in a safe state?
- What will happen that the process p2 makes one additional request for an instance of type C?

17. (a) Consider the following segment table (4) (Nov '16) (Ref.Pg.No.29 Qn.No.19)

Segment	Base	Length
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

What are the physical addresses for the following logical addresses?

(i) 0,430

(ii) 1,10

(iii) 2,500

(b) Compare the main memory organization schemes of contiguous memory allocation, pure segmentation, and pure paging with respect to the following issues: (7) (Nov '16)

(i) External fragmentation

(ii) Internal fragmentation

(iii) Ability to share code across processes

18. At any instant of time, say current state has 3 process A,B,C total 10 resources are there. Figure shows current allocation state. (Nov '18)

Process	Has	Max
A	3	9
B	2	4
C	2	7

Free resources 3 are there. Is there exist a safe sequence of allocations that allows process to complete?